

Evaluación Sumativa 2 (ES2)

Unidad 2: Framework back end · Django Admin y base de datos poblada con IA

1. Información general

Campo	Detalle
Evidencia	Aplicación web con Django Admin (proyecto publicado en GitHub)
Aprendizaje esperado	2.1 Codifica aplicaciones web para proponer soluciones tentativas a una problemática, utilizando un framework del lado del servidor y herramientas de inteligencia artificial como apoyo al desarrollo.
Criterios de evaluación	2.1.1 · 2.1.2
Ponderación	25% de la asignatura
Modalidad	Individual, presencial, en laboratorio
Duración	90 minutos
Instrumento	Escala de apreciación
Uso de IA	N2 (copiloto). Uso específico requerido: generar con IA el poblamiento inicial de la base de datos (fixture o script ORM). Toda consulta se registra en uso_ia.md.
Trazabilidad	Repositorio GitHub propio de esta evaluación, con un commit al cierre de cada etapa.
Base de datos	SQLite o MySQL según indique el docente, configurada en settings.py.

2. Proyecto transversal: Catálogo Online

Durante el semestre usted construye, por etapas, la plataforma web de un negocio local: primero la interfaz (ES1), luego la administración del contenido con base de datos (ES2), después reportes y formulario de clientes (ES3) y finalmente la API REST para integraciones (ES4).

En esta evaluación: el contenido deja de estar estático en la vista: ahora vive en una base de datos y se administra desde Django Admin. El template principal muestra el listado leyendo desde la BD.

Repositorio y punto de partida: cada evaluación se trabaja en un repositorio GitHub nuevo e independiente, creado al inicio de la prueba con el nombre apellido-ti3041-esN. Puede partir desde cero o desde el repositorio base publicado por el docente; partir desde cero incluye reutilizar el código de su propia evaluación anterior.

Personalización por sección: el docente indica la variante al inicio.

Sección	Negocio	Campos mínimos del dato principal	Volumen
A	Ferretería	nombre, categoría, precio, stock	40 productos
B	Minimarket	nombre, marca, precio, disponible	60 productos
C	Farmacia	nombre, laboratorio, precio, requiere_receta	30 productos

3. Estructura de la evaluación

El trabajo se organiza en etapas. Cada etapa termina con un commit en GitHub con el mensaje indicado. El registro de consultas a la IA se completa durante todo el desarrollo.

Etapas	Actividad	Commit
0. Entorno	Nuevo repositorio, entorno virtual y proyecto operativo (reutilizando código de la ES1 o el base del docente)	etapa-0-entorno
1. Modelo y BD	Modelo de datos, conexión a la base de datos en settings.py y migraciones aplicadas	etapa-1-modelo
2. Admin	Superusuario y modelo registrado en /admin con list_display y búsqueda o filtro	etapa-2-admin
3. Poblamiento y listado	Poblamiento inicial generado con IA cargado en la BD; el template principal lee desde la BD	etapa-3-bd
4. Entrega	Documento de uso de IA, push final y verificación del repositorio	entrega-final

4. Etapa 0: Nuevo repositorio y proyecto operativo

- Cree el repositorio apellido-ti3041-es2 y la carpeta de trabajo con venv y requirements.txt (Django; mysqlclient solo si corresponde).
- Puede partir desde cero, copiar el código de su ES1 (sin venv ni historial git) o clonar el repositorio base del docente. Verifique que levanta antes de continuar.
- Commit etapa-0-entorno.

5. Etapa 1: Modelo y conexión a la base de datos

- Configure la conexión en settings.py según lo indicado por el docente (criterio 2.1.1).
- Cree el modelo de su variante con campos y tipos adecuados, y método __str__.
- Genere y aplique migraciones (makemigrations, migrate). Commit etapa-1-modelo.

6. Etapa 2: Django Admin

- Cree el superusuario y verifique acceso a /admin.
- Registre el modelo en admin.py con list_display (3 campos o más) y search_fields o list_filter.
- Cree, edite y elimine un registro de prueba desde /admin para verificar la gestión del contenido. Commit etapa-2-admin.

7. Etapa 3: Poblamiento con IA y listado desde la BD

- **Poblamiento con IA (obligatorio):** solicite a la IA los datos de su variante (volumen completo, datos realistas en español) como fixture JSON de Django o script de poblamiento con el ORM. Cargue los datos (loaddata o ejecutando el script) y verifíquelos en /admin.
- Reemplace la lista estática de la ES1: el template principal ahora muestra el listado consultando la BD con el ORM. Queda prohibido mantener datos en duro en vistas o templates.
- Commit etapa-3-bd.

8. Documento de uso de IA (uso_ia.md)

Cree en la raíz del repositorio el archivo uso_ia.md con dos partes:

Parte 1, registro de consultas: cada consulta realizada a la IA durante la evaluación: el prompt textual, un resumen de la respuesta y qué usó o modificó antes de integrarla.

- **Incluya especialmente:** el prompt del poblamiento, la salida obtenida y los ajustes que hizo para cargarla en su BD (nombres de app, modelo, campos, formato).

Parte 2, explicación del proceso: un texto de 10 a 20 líneas donde explica CON SUS PROPIAS PALABRAS cómo usó la IA: qué le pidió y por qué, qué respuestas le sirvieron tal cual, cuáles tuvo que corregir y qué aprendió en el proceso.

No se aceptan en la Parte 2 textos redactados o resumidos por IA: la redacción debe ser personal y se contrasta en la revisión con el docente. Un texto genérico que no se condice con el trabajo del repositorio se considera no logrado.

9. Revisión personal con el docente

Al cierre de la evaluación (o durante la clase siguiente, según lo organice el docente), cada estudiante tiene una revisión individual de 5 minutos:

- Muestra su aplicación funcionando desde el repositorio entregado.
- El docente elige un fragmento del código al azar y el estudiante lo explica.
- Se contrasta uso_ia.md con el código: las consultas declaradas deben corresponder al trabajo realizado.

Esta revisión respalda los indicadores de comprensión de la escala de apreciación. No poder explicar el propio código afecta directamente esos indicadores.

10. Escala de apreciación

Niveles: L = Logrado (2 pts) · ML = Medianamente logrado (1 pt) · NL = No logrado (0 pts). Puntaje máximo: 26 pts. Exigencia 60%.

N	Indicador	Criterio	Pts
1	Nuevo repositorio operativo con venv, requirements.txt y proyecto funcionando	2.1.1	
2	Conexión a la base de datos configurada correctamente en settings.py	2.1.1	
3	Modelo con campos y tipos adecuados a la variante, con __str__ definido	2.1.1	
4	Migraciones generadas y aplicadas sin errores	2.1.1	
5	Superusuario operativo y modelo registrado en /admin	2.1.2	
6	list_display con 3 o más campos y búsqueda o filtro configurados	2.1.2	
7	Gestión demostrada desde /admin: crear, editar y eliminar se reflejan en el sitio	2.1.2	
8	Poblamiento inicial generado con IA y cargado con el volumen de la variante	2.1.1 · IA	
9	Template principal lee desde la BD con ORM, sin datos en duro	2.1.2	
10	Historial GitHub con los 5 commits de etapa y progresión coherente	Trazabilidad	
11	uso_ia.md Parte 1 completa, incluyendo el poblamiento con sus ajustes	Uso de IA	
12	uso_ia.md Parte 2 con redacción personal, coherente con el repositorio	Uso de IA	
13	Revisión personal: ejecuta su aplicación y explica el fragmento consultado	Comprensión	

Anexo (docente)

- Clonar, instalar, migrar y cargar la fixture: la aplicación debe levantar con el listado poblado.
- Solicitar credenciales de prueba del superusuario en la entrega.
- Verificar que la fixture o script de poblamiento declarado existe en el repositorio.

Prompt de ejemplo para el poblamiento (variante A):

```
Genera una fixture JSON de Django para el modelo catalogo.Producto con 40 productos de ferreteria chilenos realistas. Campos: nombre, categoria, precio (int, 1000 a 150000), stock (int, 0 a 50). Formato: lista de objetos con model, pk y fields, lista para cargar con loaddata.
python manage.py loaddata productos.json
```